



NUST CHIP DESIGN CENTRE

RISC-V Assembly

EXERCISE # 01 **RISCV Assembly Tasks**

<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Imran & Sir Umer</u></i>
<u>Date</u>	<i><u>19th Aug 2025</u></i>

1. In-Lab Tasks: (Write your lab task & screenshots here)

i. Task 1:

Question # 01

.data

.equ m1, -76

.equ m2, 5

.globl main

.text

int multiply(int a0, int a1) -> result in a2

multiply:

```
addi sp, sp, -16    # make stack frame
sw ra, 12(sp)       # save return address
sw a0, 8(sp)        # save a0
sw a1, 4(sp)        # save a1
```

```
li a2, 0            # result = 0
```

```
li t0, 0            # sign = 0
```

```
# Check sign of a0
```

```
bltz a0, negA
```

skipA:

```
j checkB
```

negA:

```
sub a0, x0, a0      # make positive
```

```
xori t0, t0, 1      # flip sign
```

checkB:

RISC-V Assembly

bltz a1, negB

skipB:

j multiply_loop

negB:

sub a1, x0, a1

xori t0, t0, 1 # flip sign

Repeated addition loop

multiply_loop:

beq a1, x0, done # while (a1 != 0)

add a2, a2, a0 # result += a0

addi a1, a1, -1

j multiply_loop

done:

beq t0, x0, end_sign # if sign == 0 skip

sub a2, x0, a2 # apply negative

end_sign:

lw ra, 12(sp) # restore ra

lw a0, 8(sp) # restore a0 (optional)

lw a1, 4(sp) # restore a1 (optional)

addi sp, sp, 16 # restore stack

ret

main:

Example inputs: a0 = -6, a1 = 5

li a0, m1 # multiplicand

li a1, m2 # multiplier

jal multiply

RISC-V Assembly

```
# Now result is in a2
# (Here you can put ecall to print if running on RARS/Venus)
mv s0, a0
mv s1, a1
mv a1, a2
li a0, 1
ecall
li a0, 10    # exit
ecall
```



ii. Task 2:

```
# Question # 02
.data
# Change values here for testing
.equ m1, -740    # dividend
.equ m2, 5       # divisor

.globl main
```

RISC-V Assembly

.text

int divide(int a0, int a1) -> quotient in a2, remainder in a3

divide:

```
addi sp, sp, -16    # make stack frame
sw ra, 12(sp)       # save return address
sw a0, 8(sp)        # save dividend
sw a1, 4(sp)        # save divisor
```

```
li a2, 0            # quotient = 0
mv a3, a0           # remainder starts as dividend
li t0, 0            # sign flag for quotient
li t1, 0            # sign flag for remainder (same as dividend)
```

Check sign of dividend (a0)

```
bltz a0, negDividend
```

skipDividend:

```
j checkDivisor
```

negDividend:

```
sub a0, x0, a0      # make dividend positive
sub a3, x0, a3      # remainder also positive
xori t0, t0, 1      # flip quotient sign
xori t1, t1, 1      # remainder has same sign as original dividend
```

checkDivisor:

```
bltz a1, negDivisor
```

skipDivisor:

```
j division_loop
```

negDivisor:

```
sub a1, x0, a1      # make divisor positive
xori t0, t0, 1      # flip quotient sign
```

Division loop: while (dividend >= divisor)

division_loop:

```
blt a0, a1, done     # if dividend < divisor -> stop
sub a0, a0, a1        # dividend -= divisor
addi a2, a2, 1        # quotient++
j division_loop
```

done:

```
mv a3, a0            # remainder = leftover dividend
```

Apply sign to quotient

```
beq t0, x0, skipQsign
```

```
sub a2, x0, a2
```

skipQsign:

RISC-V Assembly

```
# Apply sign to remainder (same sign as original dividend)
beq t1, x0, skipRsign
sub a3, x0, a3
skipRsign:
```

```
lw ra, 12(sp)    # restore ra
lw a0, 8(sp)     # restore a0 (optional)
lw a1, 4(sp)     # restore a1 (optional)
addi sp, sp, 16  # restore stack
ret
```

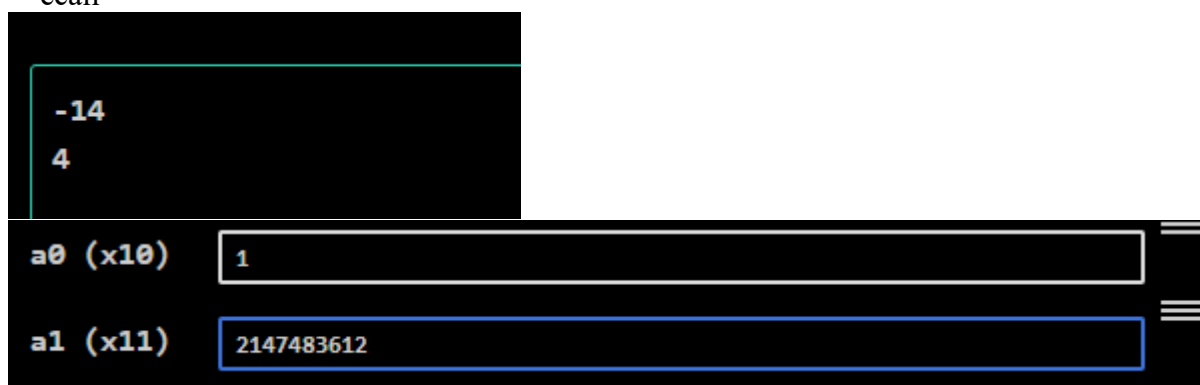
```
main:
li a0, m1        # dividend
li a1, m2        # divisor
jal divide

# Now quotient in a2, remainder in a3
# Print quotient
mv a1, a2
li a0, 1
ecall

# Print new line
addi a1, x0, '\n'
addi a0, x0, 11 # Print newline
ecall

# Print remainder
mv a1, a3
li a0, 1
ecall

li a0, 10        # exit
ecall
```



a0 (x10)	-74	
a1 (x11)	5	
a2 (x12)	-14	
a3 (x13)	4	